

Лабораторная работа № 5

Тема работы: Разработка Windows-сервиса.

Цель работы: Научиться проектировать, реализовывать и развертывать Windows-сервисы в среде операционной системы Windows.

Теоретические сведения по изучению темы

Windows-сервис – это процесс операционной системы, выполняющий служебные функции. Как правило, сервис стартует при загрузке операционной системы и заканчивает свою работу при завершении работы операционной системы. Часто сервисы называют службами.

Обычно сервисы выполняют служебные функции, необходимые для какого-нибудь приложения. Иногда сервисы используются для доступа к внешним устройствам, они называются драйверами. В виде сервисов реализованы сетевые службы, такие как DNS, FTP, Telnet, DHCP и другие. Сам сервис может быть как консольным приложением, так и приложением с графическим интерфейсом.

Для просмотра действующих на локальном компьютере Windows-сервисов может быть использована mmc-оснастка «Службы» (рис. 1).

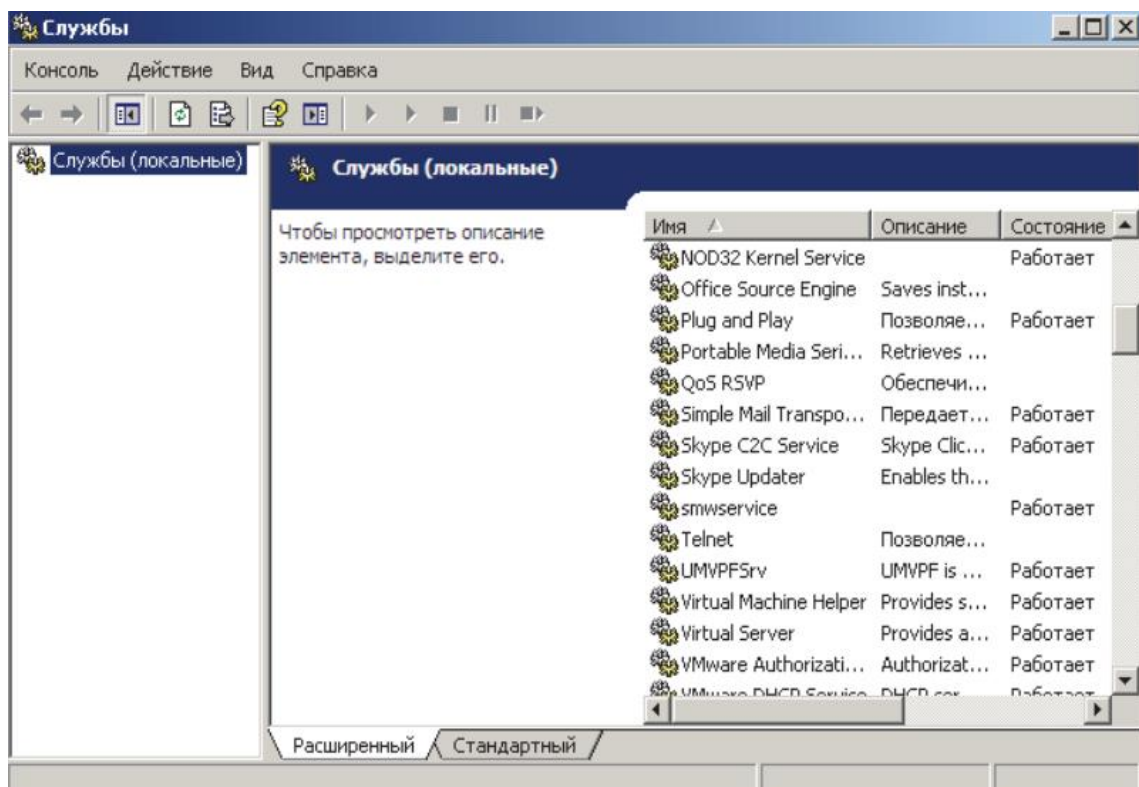


Рис. 1. Внешний вид mmc-оснастки «Службы»

Оснастка позволяет остановить или запустить работу службы, просмотреть и поменять некоторые свойства.

Принципы разработки Windows-сервиса

Обычно Windows-сервисы (далее просто сервисы) разрабатываются как консольное приложение. Приложение, как правило содержит несколько функций, предназначенных для: установки (регистрации) сервиса, запуска и остановки сервиса, для реализации сервиса.

Обычно точка входа сервиса имеет имя `ServiceMain`. Основное назначение функции `ServiceMain` – определить текущее состояние сервиса (структура `ServiceStatus` и функция `SetServiceStatus`), а также зарегистрировать обработчик управляющих команд (функция `SetServiceStatus`).

Обработчик управляющих команд – это функция, имеющая обычно имя `ServiceHandler` и предназначенная для обработки команд, поступающих сервису, например, через `mmc`-консоль. Именно эта функция определяет перечень команд, на которые реагирует сервис. Функция принимает единственный параметр, который интерпретируется как команда, которую следует выполнить. В зависимости от команды устанавливается новый статус сервиса.

Далее представлен пример реализации сервиса.

```
1. #include <Windows.h>
2. #include <iostream>
3.
4. SERVICE_STATUS      g_ServiceStatus = { 0 };
5. SERVICE_STATUS_HANDLE g_StatusHandle = NULL;
6. HANDLE              g_ServiceStopEvent = INVALID_HANDLE_VALUE;
7.
8. VOID WINAPI ServiceMain(DWORD argc, LPTSTR *argv);
9. VOID WINAPI ServiceCtrlHandler(DWORD);
10. DWORD WINAPI ServiceWorkerThread(LPVOID lpParam);
11.
12. #define SERVICE_NAME L"MySampleService"
13.
14. int wmain(int argc, LPWSTR* argv)
15. {
16.     SERVICE_TABLE_ENTRY ServiceTable[] =
17.     {
18.         { (LPWSTR)SERVICE_NAME, (LPSERVICE_MAIN_FUNCTION)ServiceMain},
19.         {NULL, NULL}
20.     };
21.
22.     if (argc == 2)
23.     {
24.         SC_HANDLE hSCManager = OpenSCManager(NULL, NULL, SC_MANAGER_CREATE_SERVICE);
25.
26.         if (!wcscmp(argv[1], L"-create"))
27.         {
28.             std::wcout << L"Creating service..." << std::endl;
29.
30.             SC_HANDLE handErr = CreateService(
31.                 hSCManager,
32.                 SERVICE_NAME,
33.                 SERVICE_NAME,
34.                 SERVICE_ALL_ACCESS,
35.                 SERVICE_WIN32_OWN_PROCESS,
36.                 SERVICE_DEMAND_START,
```

```

37.         SERVICE_ERROR_NORMAL,
38.         argv[0],
39.         NULL, NULL, NULL, NULL, NULL);
40.
41.         if (handErr == NULL)
42.         {
43.             std::wcout << L"Error of creating service!" << std::endl;
44.             return 1;
45.         }
46.         std::wcout << L"Service created!" << std::endl;
47.     }
48.     else if (!wcscmp(argv[1], L"-start"))
49.     {
50.         std::wcout << L"Starting service..." << std::endl;
51.         SC_HANDLE service = OpenService(hSCManager, SERVICE_NAME, SERVICE_START);
52.         if (!StartService(service, 0, nullptr))
53.         {
54.             std::wcout << L"Error of starting service!" << std::endl;
55.             return 2;
56.         }
57.         std::wcout << L"Service started!" << std::endl;
58.     }
59.     else if (!wcscmp(argv[1], L"-delete"))
60.     {
61.         DeleteService(OpenService(hSCManager, SERVICE_NAME, SERVICE_STOP | DELETE));
62.         std::wcout << L"Service deleted!" << std::endl;
63.     }
64.     else if (!wcscmp(argv[1], L"-stop"))
65.     {
66.         std::wcout << L"Stopping service..." << std::endl;
67.         DeleteService(OpenService(hSCManager, SERVICE_NAME, SERVICE_STOP | DELETE));
68.         SC_HANDLE handErr = CreateService(
69.             hSCManager,
70.             SERVICE_NAME,
71.             SERVICE_NAME,
72.             SERVICE_ALL_ACCESS,
73.             SERVICE_WIN32_OWN_PROCESS,
74.             SERVICE_DEMAND_START,
75.             SERVICE_ERROR_NORMAL,
76.             argv[0],
77.             NULL, NULL, NULL, NULL, NULL);
78.
79.         if (handErr == NULL)
80.         {
81.             std::wcout << L"Error of creating service!" << std::endl;
82.             return 1;
83.         }
84.         std::wcout << L"Service stopped!" << std::endl;
85.     }
86. }
87. }
88.
89. if (StartServiceCtrlDispatcher(ServiceTable) == FALSE)
90. {
91.     system("pause");
92.     return GetLastError();
93. }
94.
95. system("pause");
96. return 0;
97. }
98.
99. VOID WINAPI ServiceMain(DWORD argc, LPTSTR *argv)
100. {
101.     DWORD Status = E_FAIL;
102.
103.     // Register our service control handler with the SCM
104.     g_StatusHandle = RegisterServiceCtrlHandler(SERVICE_NAME, ServiceCtrlHandler);
105.
106.     if (g_StatusHandle == NULL)
107.         return;
108.
109.     // Tell the service controller we are starting
110.     ZeroMemory(&g_ServiceStatus, sizeof(g_ServiceStatus));
111.     g_ServiceStatus.dwServiceType = SERVICE_WIN32_OWN_PROCESS;
112.     g_ServiceStatus.dwControlsAccepted = 0;
113.     g_ServiceStatus.dwCurrentState = SERVICE_START_PENDING;

```

```

114. g_ServiceStatus.dwWin32ExitCode = 0;
115. g_ServiceStatus.dwServiceSpecificExitCode = 0;
116. g_ServiceStatus.dwCheckPoint = 0;
117.
118. if (SetServiceStatus(g_StatusHandle, &g_ServiceStatus) == FALSE)
119. {
120.     OutputDebugString(
121.         L"My Sample Service: ServiceMain: SetServiceStatus returned error");
122. }
123.
124. /*
125.  * Perform tasks necessary to start the service here
126.  */
127.
128. // Create a service stop event to wait on later
129. g_ServiceStopEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
130. if (g_ServiceStopEvent == NULL)
131. {
132.     // Error creating event
133.     // Tell service controller we are stopped and exit
134.     g_ServiceStatus.dwControlsAccepted = 0;
135.     g_ServiceStatus.dwCurrentState = SERVICE_STOPPED;
136.     g_ServiceStatus.dwWin32ExitCode = GetLastError();
137.     g_ServiceStatus.dwCheckPoint = 1;
138.
139.     if (SetServiceStatus(g_StatusHandle, &g_ServiceStatus) == FALSE)
140.     {
141.         OutputDebugString(
142.             L"My Sample Service: ServiceMain: SetServiceStatus returned error");
143.     }
144.     return;
145. }
146.
147. // Tell the service controller we are started
148. g_ServiceStatus.dwControlsAccepted = SERVICE_ACCEPT_STOP;
149. g_ServiceStatus.dwCurrentState = SERVICE_RUNNING;
150. g_ServiceStatus.dwWin32ExitCode = 0;
151. g_ServiceStatus.dwCheckPoint = 0;
152.
153. if (SetServiceStatus(g_StatusHandle, &g_ServiceStatus) == FALSE)
154. {
155.     OutputDebugString(
156.         L"My Sample Service: ServiceMain: SetServiceStatus returned error");
157. }
158.
159. // Start a thread that will perform the main task of the service
160. HANDLE hThread = CreateThread(NULL, 0, ServiceWorkerThread, NULL, 0, NULL);
161.
162. // Wait until our worker thread exits signaling that the service needs to stop
163. WaitForSingleObject(hThread, INFINITE);
164.
165.
166. /*
167.  * Perform any cleanup tasks
168.  */
169.
170. CloseHandle(g_ServiceStopEvent);
171.
172. // Tell the service controller we are stopped
173. g_ServiceStatus.dwControlsAccepted = 0;
174. g_ServiceStatus.dwCurrentState = SERVICE_STOPPED;
175. g_ServiceStatus.dwWin32ExitCode = 0;
176. g_ServiceStatus.dwCheckPoint = 3;
177.
178. if (SetServiceStatus(g_StatusHandle, &g_ServiceStatus) == FALSE)
179. {
180.     OutputDebugString(
181.         L"My Sample Service: ServiceMain: SetServiceStatus returned error");
182. }
183. }
184.
185. VOID WINAPI ServiceCtrlHandler(DWORD CtrlCode)
186. {
187.     switch (CtrlCode)
188.     {
189.     case SERVICE_CONTROL_STOP:
190.

```

```

191.         if (g_ServiceStatus.dwCurrentState != SERVICE_RUNNING)
192.             break;
193.
194.         /*
195.          * Perform tasks necessary to stop the service here
196.          */
197.
198.         g_ServiceStatus.dwControlsAccepted = 0;
199.         g_ServiceStatus.dwCurrentState = SERVICE_STOP_PENDING;
200.         g_ServiceStatus.dwWin32ExitCode = 0;
201.         g_ServiceStatus.dwCheckPoint = 4;
202.
203.         if (SetServiceStatus(g_StatusHandle, &g_ServiceStatus) == FALSE)
204.         {
205.             OutputDebugString(
206.                 L"My Sample Service: ServiceCtrlHandler: SetServiceStatus returned error");
207.         }
208.
209.         // This will signal the worker thread to start shutting down
210.         SetEvent(g_ServiceStopEvent);
211.
212.         break;
213.
214.     default:
215.         break;
216.     }
217. }
218.
219. DWORD WINAPI ServiceWorkerThread(LPVOID lpParam)
220. {
221.     // Periodically check if the service has been requested to stop
222.     while (WaitForSingleObject(g_ServiceStopEvent, 0) != WAIT_OBJECT_0)
223.     {
224.         /*
225.          * Perform main service function here
226.          */
227.
228.         // Simulate some work by sleeping
229.         Sleep(3000);
230.     }
231.
232.     return ERROR_SUCCESS;
233. }

```

Код представляет собой реализацию Windows-сервиса, созданного с использованием Windows API, который позволяет выполнять фоновые задачи в операционной системе Windows. Определены несколько глобальных переменных, включая SERVICE_STATUS для хранения текущего статуса службы, SERVICE_STATUS_HANDLE для управления этим статусом и дескриптор события g_ServiceStopEvent, который используется для сигнализации о необходимости остановки службы.

Основная точка входа в программу представлена функцией wmain, которая обрабатывает аргументы командной строки, позволяя пользователю создавать, запускать, останавливать и удалять службу с помощью функций API, таких как CreateService, StartService и DeleteService. Функция ServiceMain отвечает за регистрацию обработчика команд управления, установку начального статуса службы и создание события, на которое будет ожидать рабочий поток. Этот поток, реализованный в функции ServiceWorkerThread, периодически выполняет основную задачу службы, в то время как ServiceCtrlHandler обрабатывает команды управления, такие как запросы на остановку, изменяя статус службы и сигнализируя о завершении работы.

Задание:

- 1) Разработайте приложение, которое регистрирует, запускает, останавливает Windows-сервис, удаляет его регистрацию в зависимости от входного параметра;
- 2) Windows-сервис должен формировать и записывать каждые 10 секунд строку, содержащую текущее время в текстовый файл;
- 3) Windows-сервис должен реагировать на команды «Стоп», «Пауза», «Перезапустить», введенные через mms-консоль;
- 4) реализуйте функциональность, позволяющую отслеживать и записывать в текстовый файл информацию о том, какие команды выполняются в вашем сервисе и какие действия происходят в процессе его работы (например, «сервис создан», «сервис запущен» и т.д.).